DCC006: Organização de computadores I

Trabalho Prático #3

Professor: Daniel Fernandes Macedo e Omar Paranaiba Vilela Neto

Antes de começar seu trabalho, leia todas as instruções abaixo.

- O trabalho deve ser feito em grupos de até 3 (três alunos). **Cópias de trabalho acarretarão em devida penalização às partes envolvidas.**
- Entregas após o prazo não serão aceitas.
- O trabalho deve ser desenvolvido no Google Colab. O link de acesso ao arquivo do TP3 está disponível a seguir: TP3 - Google Colab. Cada grupo deve fazer uma cópia do arquivo para seu próprio ambiente e realizar o desenvolvimento a partir dela.
- Apenas um integrante do grupo deve submeter um único arquivo no formato **.zip**, contendo um arquivo **.ipynb** com a implementação do Simulador de Memória Cache e um **relatório**, conforme descrito na seção Documentação.
- A correção será **automatizada**, de forma que a saída do programa deve **seguir o formato apresentado nesta especificação**.

Simulador de Memória Cache

Neste trabalho, o objetivo é desenvolver um programa que funcione como um simulador de memória cache. O simulador será responsável por gerenciar uma cache de apenas um nível, processando acessos à memória RAM, preenchendo as linhas da cache conforme necessário e contabilizando a quantidade de *HITS* e *MISSES* ocorridos durante a execução. O programa deverá verificar se um determinado bloco de memória já está presente na cache (*HIT*) ou se precisa ser carregado da memória principal (*MISS*). Além disso, o simulador determinará em qual linha da cache um bloco de memória será armazenado, quando necessário.

Para simplificar o escopo do trabalho, o simulador não tratará os dados reais armazenados na memória. Em outras palavras, apenas o gerenciamento e o posicionamento das linhas na memória cache serão simulados, ignorando o conteúdo dos blocos.

Objetivo do trabalho

O principal objetivo deste trabalho é proporcionar a prática dos conceitos de **mapeamento direto**, **associatividade por conjunto** e **associatividade completa** que foram abordados durante o curso. A prática desses conceitos será essencial para compreender as diferenças entre os métodos de mapeamento e como eles afetam a eficiência e o desempenho da memória cache.

Especificação do simulador

O simulador será desenvolvido em **Python**, podendo utilizar qualquer biblioteca adicional que os alunos considerem necessária. A execução do programa deverá ocorrer no Google Colab, que será a plataforma oficial utilizada para a correção e avaliação das submissões..

O programa recebe como argumentos os seguintes parâmetros, apresentados na ordem especificada:

1. **O tamanho total da cache, em bytes**. Se tivermos uma cache de 4KB, por exemplo, iremos digitar 4096.
2. **O tamanho de cada linha, em bytes**. Uma linha de 1KB, por exemplo, seria entrada digitando-se 1024.
3. **O tamanho de cada grupo, em unidades**. Pensando em uma memória cache de 4KB, e páginas de 1KB, teremos 4 linhas. Se tivermos uma linha por grupo, teremos um sistema de mapeamento direto. Se tivermos 4 por grupo, teremos um sistema de associatividade completa.

4. **Um array com as entradas** para os acessos à memória. O formato e o conteúdo do que terá nesse array está especificado na seção "**Especificação do arquivo de entrada**" abaixo.

Parâmetros fixos do simulador

Além dos parâmetros fornecidos pela linha de comando, o simulador deve considerar os seguintes aspectos como fixos durante sua execução:

1. O espaço de endereçamento será de **32 bits**;
2. Os endereços sempre referenciam **bytes**, e não palavras;
3. A política de substituição de páginas é a **FIFO (*First In First Out*);**
4. A alocação de linhas em um conjunto seguirá a ordem. Ou seja, se tivermos um conjunto de tamanho dois, vamos primeiro armazenar o bloco na primeira linha do conjunto, e em seguida na segunda.
5. As linhas de um conjunto serão armazenadas de **forma consecutiva** na cache (uma atrás da outra).
6. Teremos **menos de mil linhas** de cache.:

Especificação do arquivo de entrada

O simulador recebe como entrada um **array de endereços de memória em formato de texto**, representando uma sequência de acessos à cache. Cada item do array contém **um único endereço hexadecimal**, com o prefixo `0x`, seguido de **exatamente 8 dígitos em letras maiúsculas**, representando um endereço de 32 bits.

Segue abaixo um exemplo válido de arquivo de entrada:

```
0xDEADBEEF
0x00000000
0x12345678
0xDEADBEEF
```

No exemplo acima, os endereços indicam que o simulador deverá acessar os blocos de memória correspondentes aos endereços 0xDEADBEEF, 0x00000000, 0x12345678 e novamente 0xDEADBEEF, nesta ordem. Esses endereços serão usados para determinar se os dados já estão presentes na cache (*HIT*) ou se será necessário carregar o bloco da memória principal (*MISS*).

Ao implementar a leitura do array, recomenda-se que o programa leia cada elemento do array como um endereço hexadecimal e o converta para um valor numérico interno para realizar os cálculos necessários.

Especificação da saída do simulador

A saída do simulador desenvolvido deverá ser salva em uma estrutura de lista de dicionários como o nome `history`, onde cada dicionário representa um acesso individual à cache, contendo as seguintes informações:

- **addr**: Endereço de memória acessado, no formato hexadecimal de 32 bits (exatamente 8 dígitos com prefixo 0x e letras maiúsculas). Esse valor corresponde ao **identificador do bloco**, obtido a partir do endereço original com os bits de deslocamento e, se aplicável, de conjuntos **ignorados**.

- **group**: Número do grupo (ou conjunto) da cache ao qual o bloco foi mapeado. Em caches com associatividade total (fully associative), este valor será sempre 0.

- **idx_line**: Índice da linha dentro do grupo utilizada para armazenar o bloco acessado.

- **result**: Resultado da operação de leitura na cache. Pode ser "HIT" se o bloco já estiver presente na cache, ou "MISS" se for necessário carregá-lo da memória principal.

Exemplo da estrutura de dado lista de dicionários:

```
history = [
   {'addr': '0x0037AB6F', 'group': 0, 'idx_line': 0, 'result': 'MISS'},
   {'addr': '0x00000000', 'group': 0, 'idx_line': 1, 'result': 'MISS'},
   {'addr': '0x00048D15', 'group': 0, 'idx_line': 2, 'result': 'MISS'},
   {'addr': '0x0037AB6F', 'group': 0, 'idx_line': 0, 'result': 'HIT'}
]
```

Exemplo de entrada e saída do simulador

O endereço da memória principal é fornecido como um valor hexadecimal de 32 bits. Ele representa uma posição na memória, mas inclui informações que não são relevantes para identificar o bloco na cache. Por exemplo: `0xDEADBEEF, 0x00000000, 0x12345678.`

As linhas da cache possuem um tamanho fixo, e os bits menos significativos do endereço correspondem à posição dentro da linha (deslocamento). Para calcular o identificador do bloco, esses bits devem ser eliminados. Se cada linha da cache tem 1KB (1024 bytes), são necessários 10 bits para endereçar qualquer posição dentro da linha. Portanto, os 10 bits menos significativos devem ser ignorados.

Exemplo:

Para `0xDEADBEEF`:
Binário: `11011110101011011011111011101111`
Ignorando os 10 bits menos significativos: `1101111010101101101111`**`0000000000`** → `0xDEADBC00`

Para `0x12345678`:
Binário: `00010010001101000101011001111000`
Ignorando os 10 bits menos significativos: `0001001000110100010101`**`0000000000`** → `0x12345400`

Para `0x00000000`:
Já possui os 10 bits menos significativos como 0, então permanece igual: `0x00000000`.

Se a cache tiver associatividade maior que 1, os bits usados para identificar o conjunto também serão ignorados. Para uma cache de associatividade total, como no exemplo fornecido, não há conjuntos, então nenhum bit adicional precisa ser descartado. Se fosse uma cache com associatividade, o número de conjuntos determinaria quantos bits adicionais seriam ignorados. Por exemplo:

- Em uma cache com 4 conjuntos, seriam necessários 2 bits para identificar os conjuntos. Esses bits também seriam ignorados junto com os 10 bits de deslocamento.

Após ignorar os bits de deslocamento e os bits do conjunto (se aplicável), o valor restante representa o identificador do bloco. Para simplificar, ele será formatado como um número hexadecimal de 32 bits, com os caracteres em maiúsculas, independentemente do número real de bits.

Exemplos finais:

`0xDEADBC00` → identificador: `0x0037AB6F`

`0x00000000` → identificador: `0x00000000`

`0x12345400` → identificador: `0x00048D15`

Abaixo está um exemplo completo de saída para uma cache de 4KB, com linhas de 1KB, associatividade completa e os endereços de entrada `0xDEADBEEF`, `0x00000000`, `0x12345678` e `0xDEADBEEF`:

```
=== Histórico de Acessos ===
      addr  group  idx_line result
0x0037AB6F      0         0   MISS
0x00000000      0         1   MISS
0x00048D15      0         2   MISS
0x0037AB6F      0         0    HIT

=== Estado Final da Cache ===
GROUP: 0, LINE: 0, ADDR: 0x0037AB6F
GROUP: 0, LINE: 1, ADDR: 0x00000000
GROUP: 0, LINE: 2, ADDR: 0x00048D15

HITs: 1
MISSes: 3
```

Documentação

É necessário enviar um relatório em formato PDF, explicando as decisões de projeto adotadas e os testes realizados. Caso tenha sido utilizada alguma ferramenta de Inteligência Artificial durante a execução do trabalho, o relatório deve incluir uma seção específica detalhando como essas ferramentas foram utilizadas. O documento também deve conter o nome completo e a matrícula de todos os integrantes do grupo.

Dicas e sugestões finais

- Não deixe o trabalho para o último dia. Procurem o monitor e coloquem suas dúvidas no fórum do moodle. Não viva perigosamente!
- Confira o formato da saída do seu programa. A correção automática não perdoa!

Outros exemplos

Exemplo 1

Parâmetros: **`4096 1024 4`**

Arquivo de entrada	Saída esperada
0x0CB886CA 0x06BC89BA 0x379CBAD1 0x28F3123B 0x06F98ED5	=== Histórico de Acessos === addr group idx_line result 0x00032E21 0 0 MISS 0x0001AF22 0 1 MISS 0x000DE72E 0 2 MISS 0x000A3CC4 0 3 MISS 0x0001BE63 0 0 MISS === Estado Final da Cache === GROUP: 0, LINE: 0, ADDR: 0x0001BE63 GROUP: 0, LINE: 1, ADDR: 0x0001AF22 GROUP: 0, LINE: 2, ADDR: 0x000DE72E GROUP: 0, LINE: 3, ADDR: 0x000A3CC4 HITs: 0 MISSes: 5

Exemplo 2

Parâmetros: **`4096 1024 4`**

Arquivo de entrada	Saída esperada
0x0CB886CA 0x06BC89BA 0x379CBAD1 0x28F3123B 0x06F98ED5 0x06F98ED8 0x06BC89BA	=== Histórico de Acessos === addr group idx_line result 0x00032E21 0 0 MISS 0x0001AF22 0 1 MISS 0x000DE72E 0 2 MISS 0x000A3CC4 0 3 MISS 0x0001BE63 0 0 MISS 0x0001BE63 0 0 HIT 0x0001AF22 0 1 HIT === Estado Final da Cache === GROUP: 0, LINE: 0, ADDR: 0x0001BE63 GROUP: 0, LINE: 1, ADDR: 0x0001AF22 GROUP: 0, LINE: 2, ADDR: 0x000DE72E GROUP: 0, LINE: 3, ADDR: 0x000A3CC4 HITs: 2 MISSes: 5

Exemplo 3

Parâmetros: **`4096 1024 4`**

Arquivo de entrada	Saída esperada
0x0CB886CA 0x06BC89BA 0x379CBAD1 0x28F3123B 0x06F98ED5 0x06F98ED8 0x17AA08A2	=== Histórico de Acessos === addr group idx_line result 0x00032E21 0 0 MISS 0x0001AF22 0 1 MISS 0x000DE72E 0 2 MISS 0x000A3CC4 0 3 MISS 0x0001BE63 0 0 MISS 0x0001BE63 0 0 HIT 0x0005EA82 0 1 MISS === Estado Final da Cache === GROUP: 0, LINE: 0, ADDR: 0x0001BE63 GROUP: 0, LINE: 1, ADDR: 0x0005EA82 GROUP: 0, LINE: 2, ADDR: 0x000DE72E GROUP: 0, LINE: 3, ADDR: 0x000A3CC4 HITs: 1 MISSes: 6

Exemplo 4

Parâmetros: **`4096 1024 4`**

Arquivo de entrada	Saída esperada
0x0CB886CA 0x06BC89BA 0x379CBAD1 0x28F3123B 0x06F98ED5 0x06F98ED8 0x17AA08A2 0x06BC89BA	=== Histórico de Acessos === addr group idx_line result 0x00032E21 0 0 MISS 0x0001AF22 0 1 MISS 0x000DE72E 0 2 MISS 0x000A3CC4 0 3 MISS 0x0001BE63 0 0 MISS 0x0001BE63 0 0 HIT 0x0005EA82 0 1 MISS 0x0001AF22 0 2 MISS === Estado Final da Cache === GROUP: 0, LINE: 0, ADDR: 0x0001BE63 GROUP: 0, LINE: 1, ADDR: 0x0005EA82 GROUP: 0, LINE: 2, ADDR: 0x0001AF22 GROUP: 0, LINE: 3, ADDR: 0x000A3CC4 HITs: 1 MISSes: 7

Exemplo 5

Parâmetros: **`4096 1024 2`**

Arquivo de entrada	Saída esperada

`0x0CB886CA`
`0x06BC89BA`
	`=== Histórico de Acessos ===`
	`addr group idx_line result`
	`0x00019710 1 0 MISS`
	`0x0000D791 0 0 MISS`
	
	`=== Estado Final da Cache ===`
	`GROUP: 0, LINE: 0, ADDR: 0x0000D791`
	`GROUP: 1, LINE: 0, ADDR: 0x00019710`
	
	`HITs: 0`
	`MISSes: 2`

Exemplo 6

Parâmetros: **`4096 1024 1`**

Arquivo de entrada	Saída esperada

`0x0CB886CA`
`0x06BC89BA`
`0x379CBAD1`
`0x28F3123B`
`0x06F98ED5`
`0x06F98ED8`
`0x17AA08A2`
	`=== Histórico de Acessos ===`
	`addr group idx_line result`
	`0x0000CB88 1 0 MISS`
	`0x00006BC8 2 0 MISS`
	`0x000379CB 2 0 MISS`
	`0x00028F31 0 0 MISS`
	`0x00006F98 3 0 MISS`
	`0x00006F98 3 0 HIT`
	`0x00017AA0 2 0 MISS`
	
	`=== Estado Final da Cache ===`
	`GROUP: 0, LINE: 0, ADDR: 0x00028F31`
	`GROUP: 1, LINE: 0, ADDR: 0x0000CB88`
	`GROUP: 2, LINE: 0, ADDR: 0x00017AA0`
	`GROUP: 3, LINE: 0, ADDR: 0x00006F98`
	
	`HITs: 1`
	`MISSes: 6`